

EECS 348 Assignment 3 – GenAI Analysis

a. GenAI programs used

1. **Google Gemma 4 12B** (model google/gemma-4-12b-qat, quantization-aware-trained weights)
2. **Prism ML Bonsai 27B** (model prism-ml/bonsai-27b)

b. How they were accessed

Both models were downloaded and run locally on my own computer in **LM Studio**.

c. Prompt (identical for both GenAIs)

Write a single-file C++ program that uses objects (classes), not standalone functions, to prioritize emails for a busy company CEO.

Requirements:

- Use a MaxHeap as the means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- Implement the MaxHeap from scratch using a list-based implementation, then use that MaxHeap to handle all the email prioritizing. Do not use pre-existing heap or priority queue code (no `std::priority_queue`, `std::make_heap`, etc.).
- The program reads commands from a test file, one command per line. The test file name is given as a command-line argument; if no argument is given, read the commands from standard input.

- Commands:
 - EMAIL <sender category>,<subject line>,<date>
Adds an email to the CEO's inbox queue. EMAIL is followed by a space; the remaining fields are comma-delimited. The sender category is one of: Boss (read first), Subordinate (read next), Peer (read next), ImportantPerson (read next), OtherPerson (read last). If there is more than one email from the same sender category, the newest email (not the oldest) is read first. The subject line may contain spaces but not commas. The date is in the format MM-DD-YYYY.

- NEXT

Display the next email for the CEO to read, without removing it, in this format:

Next email:

Sender: <sender category>
Subject: <subject line>
Date: <date>

- READ

The CEO has read the highest-priority email and dealt with it (remove it from the queue).

- COUNT

Display: There are <n> emails to read.

- The test file begins with the EMAIL commands, followed by the commands the CEO uses to read her inbox, but the order and number of commands may vary. The program must handle NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (the output is the same for both), and two READs in a row without an intervening NEXT (the highest-priority email is removed without being displayed).

- Sample test file:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

- Expected output for the sample (output blocks are separated by a blank line):

```
There are 6 emails to read.

Next email:
  Sender: Boss
  Subject: Never Mind
  Date: 01-03-2025

Next email:
  Sender: Boss
  Subject: Important
  Date: 12-20-2024

There are 4 emails to read.
- The program must compile with g++ on Linux.
```

e. Analysis

Both programs were compiled with `g++ -std=c++11 -Wall -Wextra -O2` and run against the sample test file from the assignment and my own test files. My test files cover NEXT and READ on an empty inbox, two NEXTs in a row, two READs in a row, newest-first ordering within a category across different years, Windows-style line endings, and spaces after the commas. Each test was run with the file name as an argument and with the file redirected into standard input. I checked the output against an independent checker.

Correctness

Gemma 4 12B: compiles, but only works with redirected input. The heap is correct, and with the sample file redirected into standard input the output matches the expected output exactly, including the blank lines between blocks. The edge cases (empty inbox, double NEXT, double READ) are also handled correctly.

The big bug is in `main()`. When a file name is given, the `std::ifstream` is declared inside the `if` block, so it is destroyed at the end of that block and the input pointer is left pointing at a dead stream. Given a file name, the program prints nothing. The second bug is Windows line endings: `getline` leaves the `\r` on each line, so `"NEXT\r"` never matches `"NEXT"` and every command except EMAIL is ignored.

Smaller issues: an unknown sender category is silently given OtherPerson's priority with a blank sender name. A badly formatted date leaves the `sscanf` variables uninitialized. A line that is just EMAIL makes `substr(6)` throw an uncaught exception. `sscanf` is used without including `<cstdio>`.

Bonsai 27B: does not compile. `g++` reports five errors. The heap uses `operator>` on `Email`, but only `operator<` is defined. `process()` calls `pop_back()` on a `const std::string&`. `main()` tries to assign `std::cin` to a `std::ifstream`, which isn't allowed.

Even with those fixed, the logic is wrong in several places. Commands are only handled if the line contains a space, so NEXT, READ and COUNT are all ignored. `siftUp` compares in the wrong direction, so it builds the heap backwards. Dates are compared as MM-DD-YYYY text, so 01-03-2025 sorts before 12-20-2024, which gives the wrong answer on the assignment's own sample. The subject and date are split with `rfind(',', firstComma + 1)`, which finds the first comma again, so both fields come out wrong. Every block is followed by an extra blank line, and an unknown sender category leaves the priority uninitialized. On the plus side, it does strip `\r` from each line.

Execution time

Both programs store the heap in a `std::vector`, so adding or removing an email is $O(\log n)$, and NEXT and COUNT are $O(1)$. That is the expected cost for a heap-based priority queue. The difference is in the work done per step. Gemma builds a `std::stringstream` and a temporary vector of fields for every EMAIL line, and compares two fields (category, then date) on every heap comparison. Bonsai compares date strings character by character, which is slower and also wrong. For inbox-sized inputs both finish instantly.

Space complexity

Both use $O(n)$ space for n emails, with no extra copies of the heap. Gemma stores a numeric category and a copy of the category name string in every email, plus the date as both a number and a string. Bonsai stores the sender name and date as strings plus a numeric category.

Maintainability

Gemma 4 12B

- + Three classes with clear jobs: `Email`, `MaxHeap`, and `EmailManager`.
- + `isHigherPriorityThan()` keeps the ordering rule in one readable method, and the heap has doc comments.
- – Every field of `Email` is public.
- – Each category name is typed twice in an if/else chain, with magic priority numbers.
- – `peek()` returns a non-const pointer into the heap, so callers could change an email in place and break heap order.
- – The file/stdin selection in `main()` is fragile, which is where the big bug came from.

Bonsai 27B

- + `Email`'s fields are private with getters.
- + The heap throws an exception on an empty pop or peek instead of failing silently.
- – It doesn't compile.
- – The comment on `operator<` claims MM-DD-YYYY strings compare in date order, which is wrong.
- – Using `<` in one sift and `>` in the other made the ordering inconsistent.

f. Selection and justification

I chose **Gemma 4 12B** as the base. It compiles, its heap is correct, and with redirected input it already produces the exact sample output and handles the edge cases from the assignment. Both of its real bugs are in input handling and can be fixed without touching the heap. Bonsai's program doesn't compile, and even after fixing the compile errors its command parsing, date comparison, field splitting and sift-up would all need to be rewritten, which is basically writing a new program.

g. Improvements made

Correctness

Corrections were performed with the help of Anthropic's Claude Opus 5.5.

- Fixed the big bug: the input file now stays open for the whole run. If the file can't be opened, the program prints an error and exits instead of silently waiting for keyboard input.
- Trailing `\r` and spaces are stripped from every line, so Windows-style test files work. Fields are trimmed, so `EMAIL Boss, Subject, 01-03-2025` also works.
- EMAIL lines are checked for three fields, a known sender category, and a real MM-DD-YYYY date. Bad lines and unknown commands are reported on `stderr` and skipped instead of crashing or being queued with a blank sender.
- Verified that the sample output matches exactly, that all of my test files pass with both a file argument and redirected input, and that 300 randomly generated command files match the independent checker.

Execution time

- Each email's category and date are combined into one integer key when the email is created, so every heap comparison is a single integer comparison instead of two.
- EMAIL lines are split with `find` and `substr` instead of a `std::stringstream` and a temporary vector.
- `pop()` moves the last email into the root instead of copying its strings. The heap is still $O(\log n)$ to add or remove and $O(1)$ for NEXT and COUNT, so these changes cut the work per operation.

Space complexity

- The category is stored as an index into one shared table of names instead of a copied name string in every email.
- No temporary vector of fields is created for each EMAIL line. Total space is still $O(n)$.

Maintainability

- The category names and their order are defined once in a table.
- Email's fields are private with read-only accessors, `peek()` returns a const pointer, and heap indices use `size_t`.
- Small helper methods handle trimming, date parsing, category lookup, and blank lines between blocks, and `main()` only chooses the input.
- Full prologue including revision date and revisions, block comments that say where each block came from, and comments on every line, as the rubric requires.

d. Generated code

(Placed at the end of this document because of its length.)

Gemma 4 12B (code block from its response, unmodified)

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <fstream>
#include <algorithm>

/**
 * Represents an email with priority attributes.
 * Priority is determined by:
 * 1. Category (Boss > Subordinate > Peer > ImportantPerson > OtherPerson)
 * 2. Date (Newer dates have higher priority)
 */
class Email {
public:
    int category; // Numeric value for priority comparison
    std::string categoryName;
    std::string subject;
    int dateVal; // Converted to YYYYMMDD for easy integer comparison
    std::string dateStr;

    Email(int cat, std::string name, std::string subj, int dateV, std::string dateS)
        : category(cat), categoryName(name), subject(subj), dateVal(dateV), dateStr(dateS) {}

    /**
     * Returns true if this email has higher priority than the other email.
     */
    bool isHigherPriorityThan(const Email& other) const {
        if (category != other.category) {
            return category > other.category;
        }
        return dateVal > other.dateVal;
    }
};

/**
 * MaxHeap implementation from scratch using a vector-based structure.
 * This handles the email prioritization logic.
 */
class MaxHeap {
private:
    std::vector<Email> heap;

    void siftUp(int idx) {
        while (idx > 0) {
            int parent = (idx - 1) / 2;
            if (heap[idx].isHigherPriorityThan(heap[parent])) {
                std::swap(heap[idx], heap[parent]);
                idx = parent;
            } else {
                break;
            }
        }
    }

    void siftDown(int idx) {
        int n = heap.size();
        while (true) {
            int largest = idx;
```

```

        int left = 2 * idx + 1;
        int right = 2 * idx + 2;

        if (left < n && heap[left].isHigherPriorityThan(heap[largest])) {
            largest = left;
        }
        if (right < n && heap[right].isHigherPriorityThan(heap[largest])) {
            largest = right;
        }

        if (largest != idx) {
            std::swap(heap[idx], heap[largest]);
            idx = largest;
        } else {
            break;
        }
    }
}

public:
    void push(const Email& e) {
        heap.push_back(e);
        siftUp(heap.size() - 1);
    }

    void pop() {
        if (heap.empty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            siftDown(0);
        }
    }

    Email* peek() {
        if (heap.empty()) return nullptr;
        return &heap[0];
    }

    size_t size() const {
        return heap.size();
    }
};

/**
 * Manages the CEO's inbox and processes commands.
 */
class EmailManager {
private:
    MaxHeap inbox;
    bool firstOutput = true;

    void printBlankLine() {
        if (!firstOutput) {
            std::cout << std::endl;
        }
    }
}

public:
    void processCommand(const std::string& line) {
        if (line.empty()) return;

        if (line.substr(0, 5) == "EMAIL") {
            // Format: EMAIL Category,Subject,Date
            std::string content = line.substr(6);
            std::stringstream ss(content);
            std::string segment;
            std::vector<std::string> parts;

            while (std::getline(ss, segment, ',')) {
                parts.push_back(segment);
            }
        }
    }
};

```

```

    }

    if (parts.size() >= 3) {
        std::string catStr = parts[0];
        std::string subj = parts[1];
        std::string dateStr = parts[2];

        int catVal = 0;
        std::string catName = "";

        // Map categories to priority values
        if (catStr == "Boss") { catVal = 4; catName = "Boss"; }
        else if (catStr == "Subordinate") { catVal = 3; catName = "Subordinate"; }
        else if (catStr == "Peer") { catVal = 2; catName = "Peer"; }
        else if (catStr == "ImportantPerson") { catVal = 1; catName = "ImportantPerson"; }
        else if (catStr == "OtherPerson") { catVal = 0; catName = "OtherPerson"; }

        // Parse date MM-DD-YYYY to YYYYMMDD integer
        int m, d, y;
        sscanf(dateStr.c_str(), "%d-%d-%d", &m, &d, &y);
        int dateVal = y * 10000 + m * 100 + d;

        inbox.push(Email(catVal, catName, subj, dateVal, dateStr));
    }
} else if (line == "NEXT") {
    Email* e = inbox.peek();
    if (e) {
        printBlankLine();
        std::cout << "Next email:" << std::endl;
        std::cout << "\tSender: " << e->categoryName << std::endl;
        std::cout << "\tSubject: " << e->subject << std::endl;
        std::cout << "\tDate: " << e->dateStr << std::endl;
        firstOutput = false;
    }
} else if (line == "READ") {
    inbox.pop();
} else if (line == "COUNT") {
    printBlankLine();
    std::cout << "There are " << inbox.size() << " emails to read." << std::endl;
    firstOutput = false;
}
}
};

int main(int argc, char* argv[]) {
    std::istream* input;
    if (argc > 1) {
        std::ifstream file(argv[1]);
        if (!file.is_open()) {
            // Fallback to cin if file fails to open
            input = &std::cin;
        } else {
            input = &file;
        }
    } else {
        input = &std::cin;
    }

    EmailManager manager;
    std::string line;
    while (std::getline(*input, line)) {
        manager.processCommand(line);
    }

    return 0;
}

```

Bonsai 27B (code block from its response, unmodified)

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <algorithm>
#include <stdexcept>

// Represents a single email with its metadata and calculated priority
class Email {
private:
    std::string sender;
    std::string subject;
    std::string date;
    int categoryPriority; // Higher value = higher priority

public:
    Email(const std::string& s, const std::string& subj, const std::string& dt)
        : sender(s), subject(subj), date(dt) {

        if (s == "Boss") categoryPriority = 5;
        else if (s == "Subordinate") categoryPriority = 4;
        else if (s == "Peer") categoryPriority = 3;
        else if (s == "ImportantPerson") categoryPriority = 2;
        else if (s == "OtherPerson") categoryPriority = 1;
    }

    const std::string& getSender() const { return sender; }
    const std::string& getSubject() const { return subject; }
    const std::string& getDate() const { return date; }

    // Comparison operator for MaxHeap ordering.
    // Returns true if *this* has lower priority than *other*.
    // Priority is determined first by category, then by date (newer dates are lexicographically
larger).
    bool operator<(const Email& other) const {
        if (categoryPriority != other.categoryPriority) {
            return categoryPriority < other.categoryPriority;
        }
        // MM-DD-YYYY format allows direct lexicographical comparison for chronological order.
        // Newer dates are lexicographically greater, so they will bubble to the top in a MaxHeap.
        return date < other.date;
    }
};

// List-based implementation of a MaxHeap (using a contiguous vector/array structure)
class MaxHeap {
private:
    std::vector<Email> heap;

    // Restore heap property after insertion by moving element up
    void siftUp(int idx) {
        while (idx > 0) {
            int parent = (idx - 1) / 2;
            if (heap[idx] < heap[parent]) {
                std::swap(heap[idx], heap[parent]);
                idx = parent;
            } else {
                break;
            }
        }
    }

    // Restore heap property after removal by moving element down
    void siftDown(int idx) {
        int n = static_cast<int>(heap.size());
        while (true) {
```

```

        int left = 2 * idx + 1;
        int right = 2 * idx + 2;
        int largest = idx;

        if (left < n && heap[left] > heap[largest]) {
            largest = left;
        }
        if (right < n && heap[right] > heap[largest]) {
            largest = right;
        }

        if (largest != idx) {
            std::swap(heap[idx], heap[largest]);
            idx = largest;
        } else {
            break;
        }
    }
}

public:
    void push(const Email& e) {
        heap.push_back(e);
        siftUp(static_cast<int>(heap.size() - 1));
    }

    Email pop() {
        if (heap.empty()) {
            throw std::runtime_error("Cannot pop from empty MaxHeap");
        }
        Email top = heap[0];
        heap[0] = heap.back(); // Move last element to root
        heap.pop_back();       // Remove last element
        if (!heap.empty()) {
            siftDown(0);
        }
        return top;
    }

    const Email& peek() const {
        if (heap.empty()) {
            throw std::runtime_error("Cannot peek empty MaxHeap");
        }
        return heap[0];
    }

    int size() const {
        return static_cast<int>(heap.size());
    }

    bool isEmpty() const {
        return heap.empty();
    }
};

// Manages the email queue and processes commands
class EmailManager {
private:
    MaxHeap inbox;

public:
    void process(const std::string& line) {
        // Strip trailing newline/carriage return characters
        size_t len = line.length();
        while (len > 0 && (line[len - 1] == '\n' || line[len - 1] == '\r')) {
            line.pop_back();
            --len;
        }
        if (line.empty()) return;

        // Extract command and arguments

```

```

        size_t spacePos = line.find(' ');
        std::string cmd;
        if (spacePos == std::string::npos) return;

        cmd = line.substr(0, spacePos);
        std::string args = line.substr(spacePos + 1);

        if (cmd == "EMAIL") {
            // Format: Category,Subject,Date
            size_t firstComma = args.find(',');
            size_t secondComma = args.rfind(',', firstComma + 1);

            if (firstComma != std::string::npos && secondComma != std::string::npos) {
                std::string cat = args.substr(0, firstComma);
                std::string subj = args.substr(firstComma + 1, secondComma - firstComma - 1);
                std::string dt = args.substr(secondComma + 1);

                inbox.push(Email(cat, subj, dt));
            }
        }
        else if (cmd == "NEXT") {
            if (!inbox.isEmpty()) {
                const Email& e = inbox.peek();
                std::cout << "Next email:\n";
                std::cout << "\tSender: " << e.getSender() << "\n";
                std::cout << "\tSubject: " << e.getSubject() << "\n";
                std::cout << "\tDate: " << e.getDate() << "\n\n";
            }
        }
        else if (cmd == "READ") {
            if (!inbox.isEmpty()) {
                inbox.pop();
            }
        }
        else if (cmd == "COUNT") {
            std::cout << "There are " << inbox.size() << " emails to read.\n\n";
        }
    }
};

int main(int argc, char* argv[]) {
    std::ifstream infile;

    // Read from file if argument provided, otherwise use standard input
    if (argc > 1) {
        infile.open(argv[1]);
        if (!infile.is_open()) {
            // Fallback to stdin if file cannot be opened
            infile = std::cin;
        }
    }
    else {
        infile = std::cin;
    }

    EmailManager manager;
    std::string line;

    while (std::getline(infile, line)) {
        manager.process(line);
    }

    if (infile != stdin) {
        infile.close();
    }

    return 0;
}

```